MODULE 01 · PYTHON + ASYNC + FASTAPI FOR LLM ENGINEERING · DAYS 06–08

Python OOP, Pydantic & Databases: Days 06–08

Object-oriented programming from the ground up — how the variables and functions you know become attributes and methods, what an object is in memory — then inheritance, abstract classes, Pydantic validation and safe database access, all with the notebooks' own examples and real output.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Notebooks: 06_OOP_Classes · 07_OOP_Inheritance_ABC · 08_Pydantic_Database
Examples: **Employee**, **Customer**, **Order** and **Support Ticket** for ShopSmart. Python 3.10+ · Pydantic 2 · PostgreSQL + psycopg 3

Start here

1 How Days 06–08 fit together

Day 06 · OOP Part 1: Classes and Objects

2 You have been using objects since Day 01
3 Where we start: separate variables and functions
4 Step 1: group each employee's data in a dict
5 Step 2: put the data and the functions together — a class
6 Class vs object
7 What happens in memory when you create an object
8 Each object has its own attributes
9 Two names, one object
10 __init__ and self, in the notebook
11 Instance methods
12 Objects remember: a Customer with a running total
13 Class attributes: one value shared by all objects
14 __repr__: a readable print-out
15 @property: calculated values that look like attributes
16 Working with a list of objects
17 Common beginner errors
18 Day 06 recap and practice

Day 07 · OOP Part 2: Inheritance and ABCs

19 Inheritance: reuse what is shared
20 What super() does to the object
21 Overriding and polymorphism
22 ABC: a class that sets rules
23 A shared contract: customer notifications
24 Public, _internal and __mangled names
25 @classmethod and @staticmethod
26 Composition: HAS-A instead of IS-A
27 isinstance(): check an object's type
28 Day 07 recap and practice

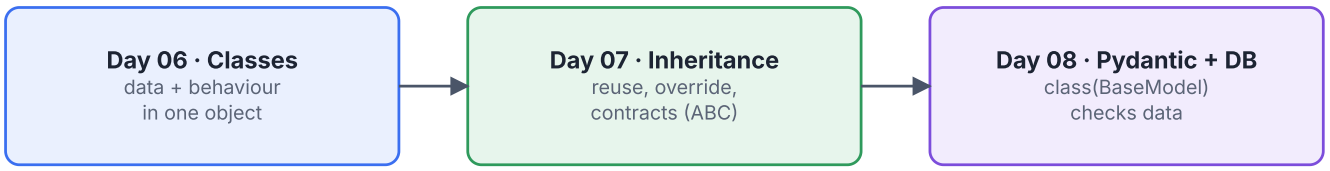
Day 08 · Pydantic and Databases

29 Why validate data?
30 Your first model
31 Add rules with Field
32 Optional fields and allowed values
33 Validate dicts and JSON, and convert back
34 AI use case: validate an LLM's classification
35 Nested models
36 Database basics: where Pydantic fits
37 Setting up PostgreSQL (optional)
38 Connect, create a table, insert a row
39 Parameters, not f-strings: SQL injection
40 Read a row and validate it
41 Day 08 recap and practice

Wrap-up · Days 06–08 in one script

42 The script
43 Read it like this

1 How Days 06–08 fit together



Days 01–05 (variables, functions, dicts, JSON, errors) are the raw material for all three

Figure 0.1 Each day builds on the one before. Day 08's Pydantic models are Day 06 classes that use Day 07 inheritance.

Day	Question it answers	You will be able to
06	How do I keep data and the code that uses it together?	Write classes, create objects, explain `self` and memory
07	How do classes share and enforce behaviour?	Use inheritance, `super()`, ABCs, composition
08	How do I trust data from APIs, LLMs and databases?	Validate with Pydantic; query PostgreSQL safely

Examples come from the notebooks (grey label = notebook cell). Where an idea needed one more example — especially objects in memory — an **extra example** was written and run; its label says so. Blue = code, green = real output. Memory addresses like `0x7f60a5482cf0` will be different on your machine.

DAY 06

OOP Part 1: Classes and Objects

Object-oriented programming (OOP) is new for most people, so this part starts from what you already know — variables and functions — and shows step by step how they become attributes and methods inside a class, and what an object really is in your computer's memory.

Notebook: **06_OOP_Classes.ipynb**

You already use objects | Variables + functions | Grouping in a dict | The class | Class vs object | Objects in memory | self | Methods | State | Class attributes | __repr__ | @property | Lists of objects | Common errors

2 You have been using objects since Day 01

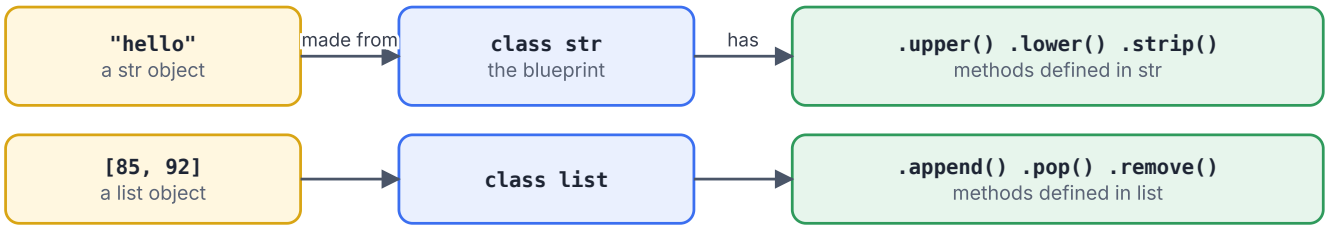


Figure 6.1 Every value you have used since Day 01 is an object made from a class.

Before writing our own classes, notice that you already use them. `"hello"` is an **object** of the class `str`. `.upper()` is a **method** — a function that belongs to that class. On Day 03 you called `numbers.append(2)`: `numbers` is a list object and `append` is a method of the `list` class.

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
text = "hello"
print(type(text))          # str is a class
print(text.upper())        # upper() is a method of the str class

scores = [85, 92]
scores.append(78)          # append() is a method of the list class
print(type(scores), scores)
```

OUTPUT

```
<class 'str'>
HELLO
<class 'list'> [85, 92, 78]
```

Today you learn to write your own class — `Employee`, `Customer` — the same way Python's authors wrote `str` and `list`.

3 Where we start: separate variables and functions

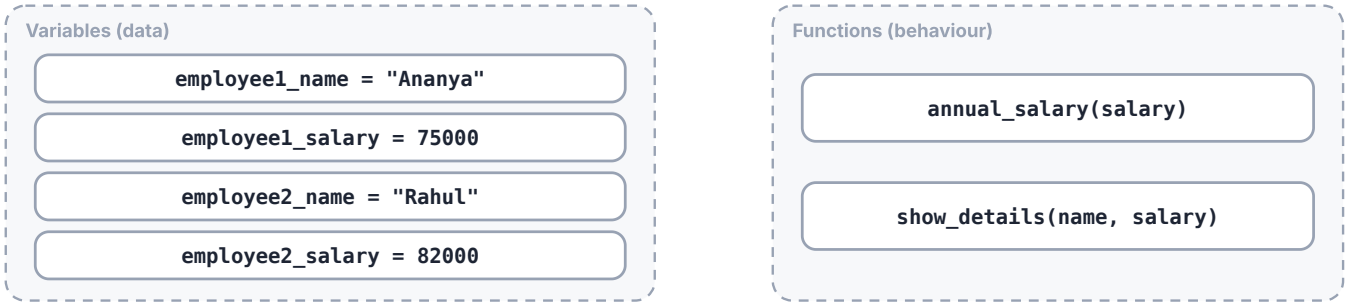


Figure 6.2 Data and the functions that use it live apart. Nothing ties a name to its salary.

With what you know from Days 01 and 03, two employees need four variables and two functions. Every call must pass the right pieces in the right order — and nothing stops you mixing them up:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
employee1_name = "Ananya"
employee1_salary = 75000
employee2_name = "Rahul"
employee2_salary = 82000

def annual_salary(salary):
    return salary * 12

def show_details(name, salary):
    print(f"{name}: ₹{salary:,} a month, ₹{annual_salary(salary):,} a year")

show_details(employee1_name, employee1_salary)
show_details(employee1_name, employee2_salary)   # mistake: Ananya's name, Rahul's salary
```

OUTPUT

```
Ananya: ₹75,000 a month, ₹900,000 a year
Ananya: ₹82,000 a month, ₹984,000 a year
```

The second line is wrong — Ananya with Rahul's salary — and Python can't notice. With 200 employees and 10 fields each, this gets out of hand quickly.

4 Step 1: group each employee's data in a dict

Day 01 gave us a better tool: one dict per employee keeps the name and salary together, so they can't be mixed up. The function now takes one employee:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
employee1 = {"name": "Ananya", "salary": 75000}
employee2 = {"name": "Rahul", "salary": 82000}

def show_details(employee):
    print(f"{employee['name']}: ₹{employee['salary']:,} a month")

show_details(employee1)
show_details(employee2)
```

OUTPUT

```
Ananya: ₹75,000 a month
Rahul: ₹82,000 a month
```

Better	Still a problem
Name and salary travel together	`show_details` is still a separate function that must be given the dict
One argument instead of two	Nothing checks the keys: `{"nme": "Ananya"}` is accepted and fails later
	There is no single place that says "an employee has a name and a salary"

5 Step 2: put the data and the functions together — a class

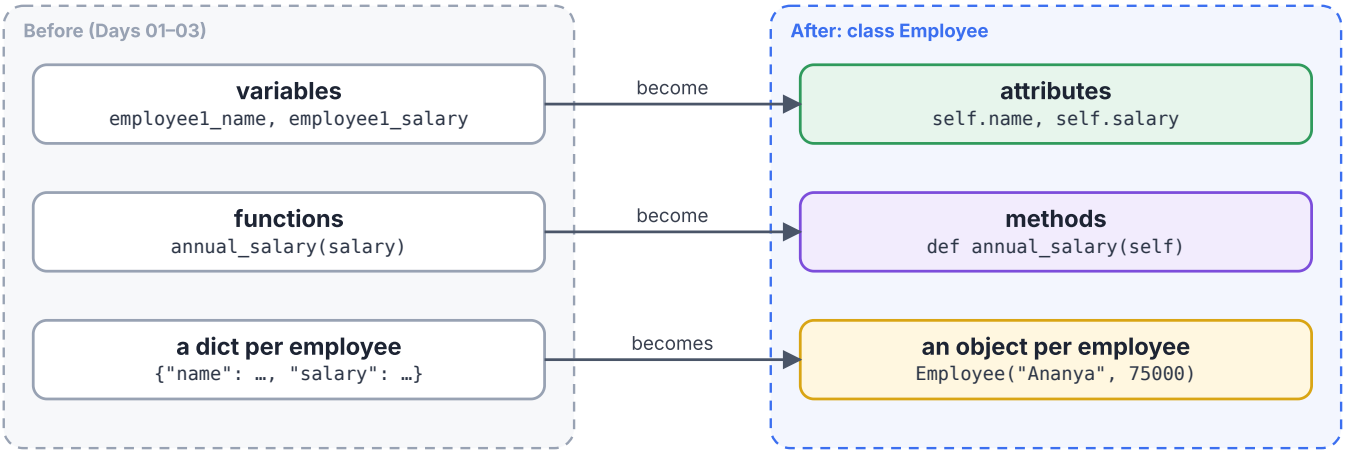


Figure 6.3 A class puts the data (attributes) and the functions (methods) in one place.

A **class** describes in one place what an employee **has** (data) and what it can **do** (functions). Inside a class we give them new names:

What you know	Inside a class it is called	Example
a variable	an **attribute**	`employee1_name` → `employee1.name`
a function	a **method** (a function defined inside the class)	`annual_salary(salary)` → `employee1.annual_salary()`
a dict for one employee	an **object** (also called an instance)	`{"name": "Ananya"}` → `Employee("Ananya", 75000)`

What you know	Inside a class it is called	Example
the dict's keys	the attribute names	`employee["name"]` → `employee.name`
passing the data in as an argument	`self` — the object the method works on	`show_details(employee1)` → `employee1.show_details()`

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Employee:
    def __init__(self, name, salary):
        self.name = name                # variable  → attribute
        self.salary = salary

    def annual_salary(self):            # function  → method
        return self.salary * 12

    def show_details(self):
        print(f"{self.name}: ₹{self.salary:,} a month, ₹{self.annual_salary():,} a year")

employee1 = Employee("Ananya", 75000)
employee2 = Employee("Rahul", 82000)

employee1.show_details()
employee2.show_details()
```

OUTPUT

```
Ananya: ₹75,000 a month, ₹900,000 a year
Rahul: ₹82,000 a month, ₹984,000 a year
```

Read `employee1.show_details()` as "employee1, show your details". The dot connects an object with its attributes and methods. The method already knows the name and salary — they belong to the object.

6 Class vs object

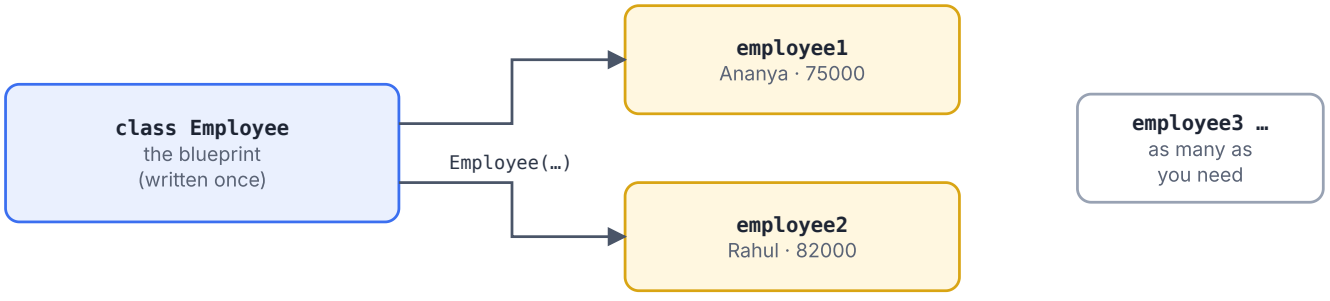


Figure 6.4 One class, many objects. Each object is a separate thing with its own values.

A class is a **blueprint**: it describes what every employee looks like. An **object** is one employee made from that blueprint. You write the class once and create as many objects as you need by calling the class like a function: `Employee()`.

NOTEBOOK CELL 5

```
class Employee:
    pass

employee1 = Employee()
employee2 = Employee()

print(type(employee1))
print(employee1 is employee2)  # Different objects
```

OUTPUT

```
<class '__main__.Employee'>
False
```

Output	Meaning
`<class '__main__.Employee'>`	`employee1` is an `Employee`. `__main__` is Python's name for the file or notebook you are running
`False`	`is` asks "the very same object?" — no, two separate objects were created

`pass` means "nothing here yet". An empty class is legal, but it has no attributes or methods — the next sections fill it in.

7 What happens in memory when you create an object

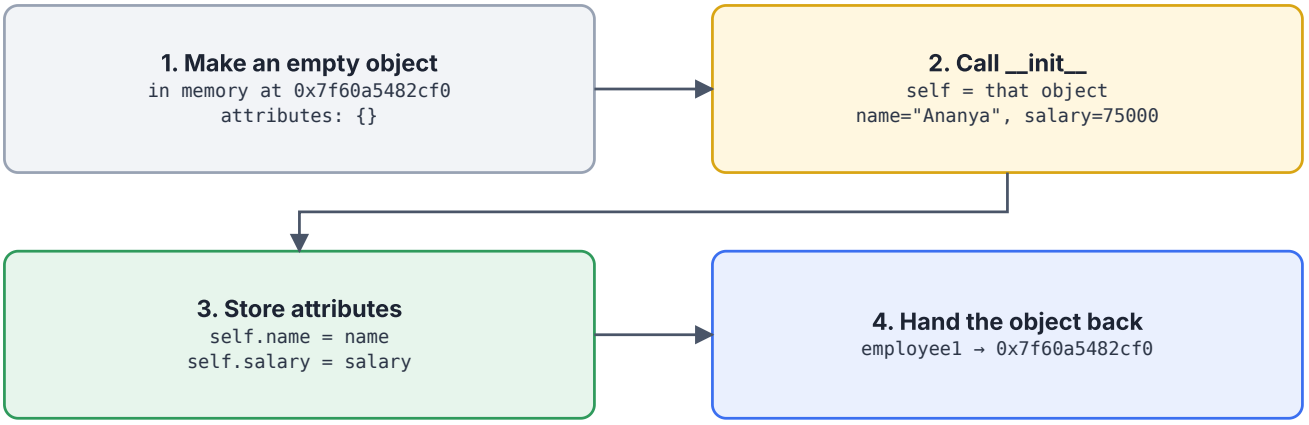


Figure 6.5 What Python does for employee1 = Employee("Ananya", 75000). The address is from a real run; yours will differ.

Your computer's memory is like a huge row of numbered boxes. Every object lives somewhere in it, at an **address**. `id(x)` gives that address as a number and `hex()` prints it the usual way (`0x…`). `__dict__` shows the attributes stored inside an object.

This version of `Employee` prints what happens inside `__init__`:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Employee:
    def __init__(self, name, salary):
        print(f"  __init__ runs. self is the object at {hex(id(self))}")
        print(f"  attributes before: {self.__dict__}")
        self.name = name
        self.salary = salary
        print(f"  attributes after : {self.__dict__}")

print("Creating employee1 ...")
employee1 = Employee("Ananya", 75000)
print("employee1 points to   ", hex(id(employee1)))
```

OUTPUT

```
Creating employee1 ...
  __init__ runs. self is the object at 0x7f60a5482cf0
  attributes before: {}
  attributes after : {'name': 'Ananya', 'salary': 75000}
employee1 points to    0x7f60a5482cf0
```

1. **The object exists before __init__ runs.** Python first makes an empty object (attributes `{}`).
2. **__init__ fills it.** `self` is that new object — the same address as the one `employee1` gets at the end.
3. **self.name = name** copies the value passed in into the object, under the name `name`.
4. **The variable is a name tag.** `employee1` does not contain the object; it points to the object's address — just like `a = 10` on Day 01.

`__init__` does not create the object — Python has already done that. `__init__` only **initialises** it (puts the first values in). That is why it is called init, not create.

8 Each object has its own attributes

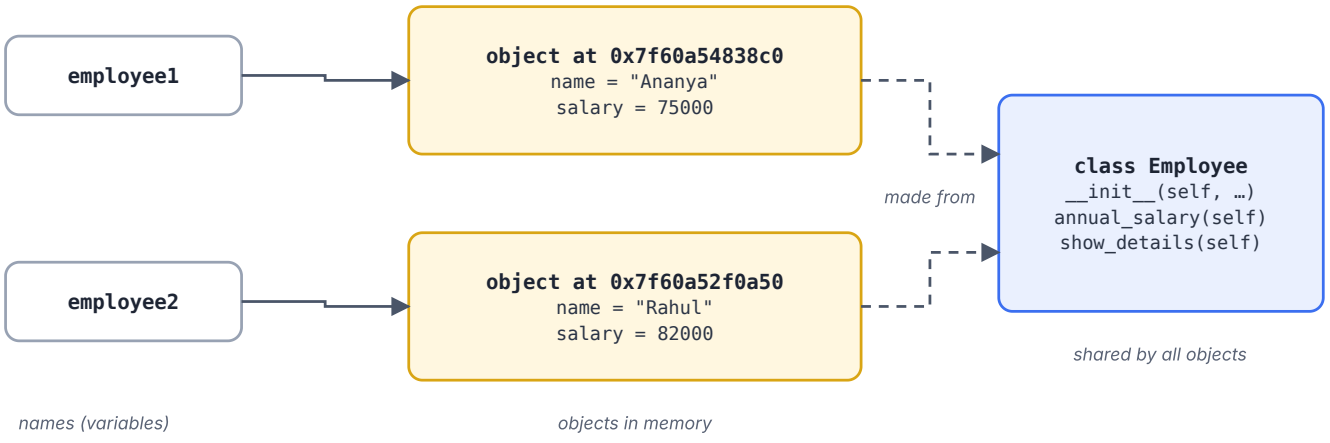


Figure 6.6 Two names, two objects, one class. Attributes live in each object; methods live once in the class.

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

employee1 = Employee("Ananya", 75000)
employee2 = Employee("Rahul", 82000)

print(hex(id(employee1)), employee1.__dict__)
print(hex(id(employee2)), employee2.__dict__)
print("Same object?", employee1 is employee2)
```

OUTPUT

```
0x7f60a54838c0 {'name': 'Ananya', 'salary': 75000}
0x7f60a52f0a50 {'name': 'Rahul', 'salary': 82000}
Same object? False
```

Two calls to `Employee(...)` make two objects at two addresses, each with its **own** `name` and `salary`. Changing `employee2.salary` can never touch `employee1`.

9 Two names, one object

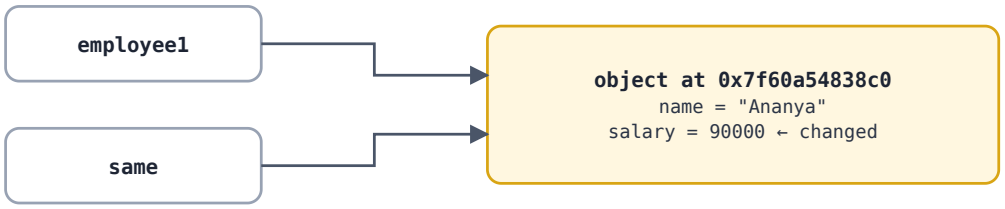


Figure 6.7 same = employee1 copies the arrow, not the object. Both names now reach one object.

Assigning an object to another variable does **not** copy it. It gives the same object a second name:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
same = employee1            # no new object: a second name for the same one
same.salary = 90000

print(employee1.salary)     # 90000 — changed through the other name
print(same is employee1)    # True
print(employee2.salary)     # 82000 — a different object, untouched
```

OUTPUT

```
90000
True
82000
```

This explains surprises later: pass an object into a function, the function changes an attribute, and the change is visible outside — because it is the same object. Day 03's lists behave the same way.

10 __init__ and self, in the notebook

Now the notebook's version. `__init__` takes the values for a new employee and stores them as **instance attributes** — attributes that belong to one object (one instance).

NOTEBOOK CELL 7

```
class Employee:
    def __init__(self, name, salary, department):
        self.name = name
        self.salary = salary
        self.department = department

employee1 = Employee("Ananya", 75000, "Data")
employee2 = Employee("Rahul", 82000, "AI")

print(employee1.name, employee1.salary, employee1.department)
print(employee2.name, employee2.salary, employee2.department)
```

OUTPUT

```
Ananya 75000 Data
Rahul 82000 AI
```

Part	Meaning
`def __init__(self, name, salary, department):`	Runs automatically when you write `Employee(...)`
`self`	The object being set up. Always the first parameter; Python fills it in
`name, salary, department`	The values you pass: `Employee("Ananya", 75000, "Data")`
`self.name = name`	Left: the attribute on the object. Right: the parameter value. Same word, two different things

self is simply the object

When you call a method on an object, Python passes that object in as `self`. These two lines do exactly the same:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def annual_salary(self):
        return self.salary * 12

employee1 = Employee("Ananya", 75000)

print(employee1.annual_salary())            # what you write
print(Employee.annual_salary(employee1))    # what Python does: the object goes in as self

print("annual_salary" in employee1.__dict__)   # False: methods are not copied into objects
print("annual_salary" in Employee.__dict__)    # True: they live once, in the class
```

OUTPUT

```
900000
900000
False
True
```

`employee1.annual_salary()` is turned into `Employee.annual_salary(employee1)`. That is why every method needs `self` as its first parameter, and why you never pass it yourself. The last two lines show methods are stored once in the class, not copied into every object.

11 Instance methods

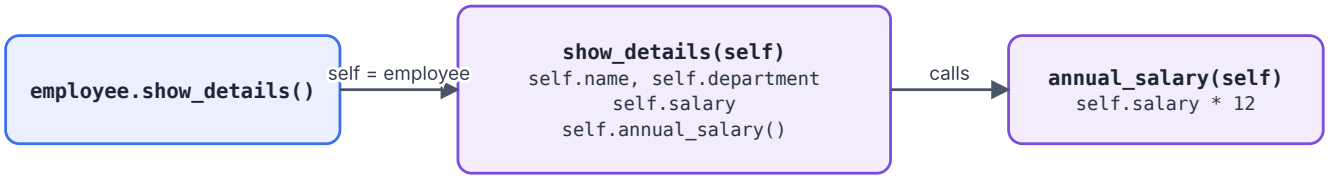


Figure 6.8 employee.show_details() — the method reads the object's attributes through self and calls another method.

A method is a function inside a class. Through `self` it can read the object's attributes and call the object's other methods — no arguments needed:

NOTEBOOK CELL 10

```
class Employee:
    def __init__(self, name, salary, department):
        self.name = name
        self.salary = salary
        self.department = department

    def annual_salary(self):
        return self.salary * 12

    def show_details(self):
        print(f"{self.name} works in {self.department}.")
        print(f"Monthly salary: ₹{self.salary:,}")
        print(f"Annual salary: ₹{self.annual_salary():,}")

employee = Employee("Ananya", 75000, "Data")
employee.show_details()
```

OUTPUT

```
Ananya works in Data.
Monthly salary: ₹75,000
Annual salary: ₹900,000
```

Inside the class write `self.salary` and `self.annual_salary()`. Outside, write `employee.salary` and `employee.annual_salary()`. Same thing — `self` is the inside name for the object.

12 Objects remember: a Customer with a running total

Attributes can change after the object is created. `total_spend` starts at 0 and each `place_order` adds to it — the object remembers its own history:

NOTEBOOK CELL 12

```
class Customer:
    def __init__(self, name, email):
        self.name = name
        self.email = email
        self.total_spend = 0

    def place_order(self, amount):
        if amount <= 0:
            print("Order amount must be positive.")
            return
        self.total_spend += amount
        print(f"Order placed for ₹{amount:,}")

    def show_summary(self):
        print(f"Customer: {self.name}")
        print(f"Email: {self.email}")
        print(f"Total spend: ₹{self.total_spend:,}")

customer = Customer("Meera", "meera@example.com")
customer.place_order(1200)
customer.place_order(800)
customer.show_summary()
```

OUTPUT

```
Order placed for ₹1,200
Order placed for ₹800
Customer: Meera
Email: meera@example.com
Total spend: ₹2,000
```

Step	Call	total_spend after
1	`Customer("Meera", "meera@example.com")`	0 (set in `__init__`)
2	`place_order(1200)`	1200
3	`place_order(800)`	2000
—	`place_order(-50)` would print the warning and `return` early	unchanged

The method guards the data: an order of 0 or less is refused, so `total_spend` can never go wrong through `place_order`. Keeping the rules next to the data is a key benefit of classes.

13 Class attributes: one value shared by all objects

Not in the notebook, but you will meet it soon. A variable written directly inside the class (not on `self`) belongs to the class, so every object sees the same value:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Employee:
    company = "ShopSmart"          # class attribute: one value, shared by all objects

    def __init__(self, name):
        self.name = name           # instance attribute: each object has its own

a = Employee("Ananya")
b = Employee("Rahul")
print(a.name, b.name)
print(a.company, b.company)

Employee.company = "ShopSmart India"   # change it once, in the class
print(a.company, b.company)
print(a.__dict__)                      # company is not stored in the object
```

OUTPUT

```
Ananya Rahul
ShopSmart ShopSmart
ShopSmart India ShopSmart India
{'name': 'Ananya'}
```

	Instance attribute	Class attribute
Written as	`self.name = name` inside `__init__`	`company = "ShopSmart"` in the class body
Stored in	each object	the class, once
Use for	values that differ per object	values that are the same for all

14 __repr__: a readable print-out

Print an object without `__repr__` and Python shows only its class and memory address — not very helpful:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Employee:
    def __init__(self, employee_id, name):
        self.employee_id = employee_id
        self.name = name

print(Employee(101, "Ananya"))
```

OUTPUT

```
<__main__.Employee object at 0x7f60a5483cb0>
```

Define `__repr__` to return a string describing the object. `print()` and Jupyter use it automatically:

NOTEBOOK CELL 14

```
class Employee:
    def __init__(self, employee_id, name, department):
        self.employee_id = employee_id
        self.name = name
        self.department = department

    def __repr__(self):
        return f"Employee(id={self.employee_id}, name='{self.name}',
department='{self.department}')"

employee = Employee(101, "Ananya", "Data")
print(employee)
```

OUTPUT

```
Employee(id=101, name='Ananya', department='Data')
```

Methods with double underscores on both sides (`__init__` , `__repr__`) are **special methods**. You don't call them yourself; Python calls them at the right moment.

15 @property: calculated values that look like attributes

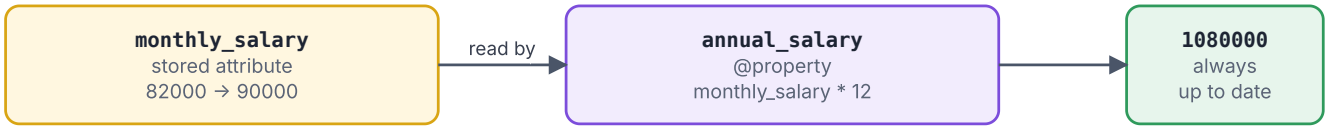


Figure 6.9 monthly_salary is stored. annual_salary is worked out fresh every time you read it.

If a value can be worked out from other attributes, don't store it — it would go out of date. `@property` turns a method into something you read like an attribute, **without** brackets:

NOTEBOOK CELL 16

```
class Employee:
    def __init__(self, name, monthly_salary):
        self.name = name
        self.monthly_salary = monthly_salary

    @property
    def annual_salary(self):
        return self.monthly_salary * 12

employee = Employee("Rahul", 82000)
print(employee.annual_salary)  # No parentheses
employee.monthly_salary = 90000
print(employee.annual_salary)  # Recalculated automatically
```

OUTPUT

```
984000
1080000
```

82000 × 12 = 984000. After the salary changes to 90000, reading `annual_salary` again gives 1080000 — no extra code.

16 Working with a list of objects

On Day 01 a table was a list of dicts. With a class it is a list of objects — the same loops and filters, but with the dot instead of square brackets:

NOTEBOOK CELL 18

```
class Employee:
    def __init__(self, name, department, salary):
        self.name = name
        self.department = department
        self.salary = salary

employees = [
    Employee("Ananya", "Data", 75000),
    Employee("Rahul", "AI", 82000),
    Employee("Meera", "Data", 79000),
]

for employee in employees:
    print(f"{employee.name}: {employee.department}, ₹{employee.salary:,}")

print("\nEmployees in Data:")
for employee in employees:
    if employee.department == "Data":
        print(employee.name)

average_salary = sum(e.salary for e in employees) / len(employees)
print(f"Average monthly salary: ₹{average_salary:,.0f}")
```

OUTPUT

```
Ananya: Data, ₹75,000
Rahul: AI, ₹82,000
Meera: Data, ₹79,000

Employees in Data:
Ananya
Meera
Average monthly salary: ₹78,667
```

	List of dicts (Day 01)	List of objects (today)
Read a value	`employee["salary"]`	`employee.salary`
Typo in the name	`KeyError` only when that line runs	`AttributeError` only when that line runs
Behaviour	separate functions	methods on each object
Checks on the data	none	possible in `__init__` and methods (and Day 08's Pydantic does it for you)

`sum(e.salary for e in employees)` is like a list comprehension without the square brackets: it adds up the salaries one by one. (75000 + 82000 + 79000) / 3 = 78666.67, printed as ₹78,667 by `:,.0f`.

17 Common beginner errors

These four mistakes cause most early OOP errors. Each was run for real:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Employee:
    def greet():                # forgot self
        print("Hello")

Employee().greet()
```

ERROR

```
TypeError: Employee.greet() takes 0 positional arguments but 1 was given
```

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def annual_salary(self):
        return salary * 12          # forgot self.

Employee("Ananya", 75000).annual_salary()
```

ERROR

```
NameError: name 'salary' is not defined
```

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Employee:
    def __init__(self, name):
        self.name = name

employee = Employee("Ananya")
print(employee.age)
```

ERROR

```
AttributeError: 'Employee' object has no attribute 'age'
```

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
employee = Employee             # forgot ()  → this is the class, not an object
print(employee)
print(employee.name)
```

OUTPUT AND ERROR

```
<class '__main__.Employee'>
AttributeError: type object 'Employee' has no attribute 'name'
```

Message	Cause	Fix
`takes 0 positional arguments but 1 was given`	Method has no `self`; Python still passes the object	`def greet(self):`

Message	Cause	Fix
`NameError: name 'salary' is not defined`	Used `salary` instead of `self.salary` inside a method	Write `self.` before attributes
`'Employee' object has no attribute 'age'`	That attribute was never set (typo or not in `__init__`)	Check the spelling; set it in `__init__`
`type object 'Employee' has no attribute …`	Used the class `Employee` instead of an object `Employee(...)`	Add the brackets and arguments

18 Day 06 recap and practice

Concept	Remember
Class	A blueprint: what objects have (attributes) and do (methods)
Object / instance	One thing made from the class: `Employee("Ananya", 75000)`
Attribute	A variable that belongs to an object: `employee.name`
Method	A function inside the class; called as `employee.annual_salary()`
`__init__`	Runs when the object is created; fills in the first attribute values
`self`	The object the method is working on; Python passes it automatically
Memory	Each object has its own address and attributes; variables point to objects; `b = a` shares, not copies
Class attribute	Defined in the class body; shared by every object
`__repr__`	What `print(object)` shows
`@property`	A calculated value read without brackets

Practice (from the notebook)

1. Create a `Customer` class with `name` and `city`; create two customers.
2. Add a `greet()` method that prints a personalised greeting.
3. Add a `purchase(amount)` method that increases `total_spend`.
4. Create three employees and print only those earning more than ₹75,000.
5. Add an `@property` called `annual_salary` to `Employee`.

DAY 07

OOP Part 2: Inheritance and ABCs

Classes can build on other classes. A child class reuses its parent's attributes and methods and adds or changes what it needs. An abstract base class (ABC) sets rules that every child must follow.

Notebook: **07_OOP_Inheritance_ABC.ipynb**

Inheritance | super() | Overriding | Polymorphism | ABC | Shared contract | _ and __ names | @classmethod | @staticmethod | Composition | isinstance

19 Inheritance: reuse what is shared

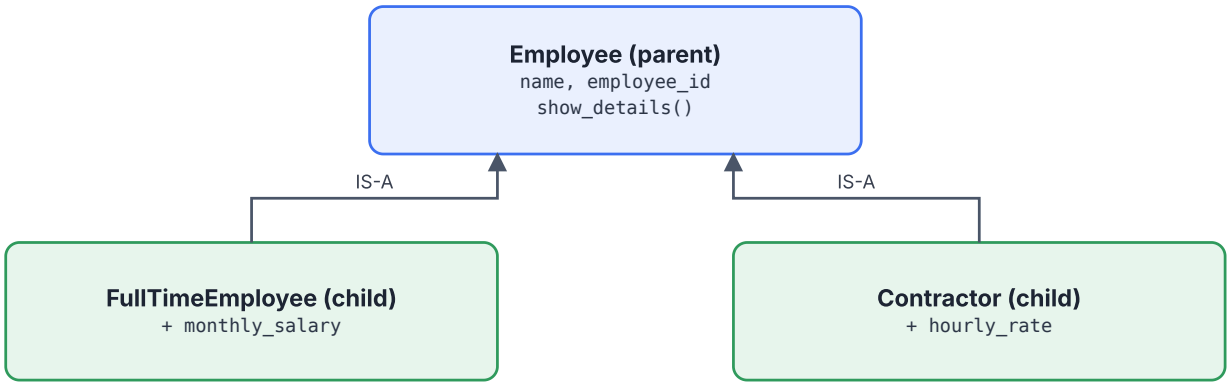


Figure 7.1 The parent holds what everyone shares. Each child adds only what is different.

Every employee has a name and an ID. A full-time employee also has a monthly salary; a contractor has an hourly rate. Instead of writing `name` and `employee_id` in both classes, write them once in a **parent** class and let the **child** classes inherit them. `class FullTimeEmployee(Employee):` means "FullTimeEmployee is built on Employee".

NOTEBOOK CELL 2

```
class Employee:
    def __init__(self, name, employee_id):
        self.name = name
        self.employee_id = employee_id

    def show_details(self):
        print(f"{self.name} (ID: {self.employee_id})")

class FullTimeEmployee(Employee):
    def __init__(self, name, employee_id, monthly_salary):
        super().__init__(name, employee_id)
        self.monthly_salary = monthly_salary

class Contractor(Employee):
    def __init__(self, name, employee_id, hourly_rate):
        super().__init__(name, employee_id)
        self.hourly_rate = hourly_rate

employee = FullTimeEmployee("Ananya", 101, 75000)
contractor = Contractor("Rahul", 201, 1200)

employee.show_details()
print("Monthly salary:", employee.monthly_salary)
contractor.show_details()
print("Hourly rate:", contractor.hourly_rate)
```

OUTPUT

```
Ananya (ID: 101)
Monthly salary: 75000
Rahul (ID: 201)
Hourly rate: 1200
```

Word	Meaning	Here
Parent / base class	Holds shared data and behaviour	`Employee`
Child / subclass	Inherits everything from the parent and adds its own	`FullTimeEmployee`, `Contractor`
`super().__init__(…)`	Runs the parent's `__init__` to set up the shared attributes	sets `name`, `employee_id`
IS-A	The test for inheritance: "a full-time employee **is an** employee"	true → inheritance fits

`contractor.show_details()` works although `Contractor` never defines it — it is inherited from `Employee`.

20 What super() does to the object

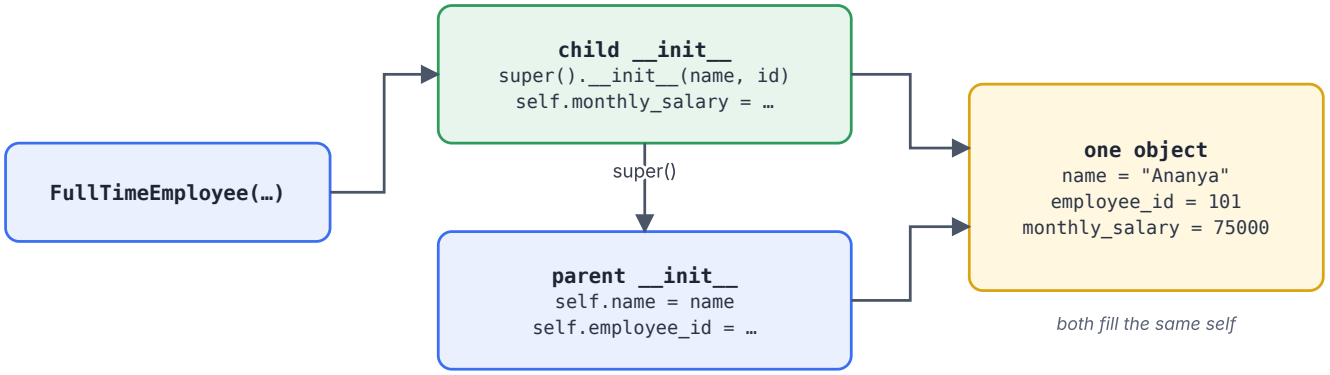


Figure 7.2 FullTimeEmployee("Ananya", 101, 75000): two __init__ methods fill ONE object.

A common worry: "does inheritance create a parent object and a child object?" No — there is **one** object. The child's `__init__` hands the same `self` to the parent's `__init__` with `super()`, and both put their attributes into it. (In this extra example the parent is called `Person`.)

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Person:
    def __init__(self, name, employee_id):
        self.name = name
        self.employee_id = employee_id

    def show_details(self):
        print(f"{self.name} (ID: {self.employee_id})")

class FullTimeEmployee(Person):
    def __init__(self, name, employee_id, monthly_salary):
        super().__init__(name, employee_id)     # parent fills name, employee_id
        self.monthly_salary = monthly_salary    # child adds its own

employee = FullTimeEmployee("Ananya", 101, 75000)
print(employee.__dict__)                        # ONE object, all three attributes
print([c.__name__ for c in FullTimeEmployee.__mro__])   # where Python looks, in order
employee.show_details()                         # not in the child → found in the parent
```

OUTPUT

```
{'name': 'Ananya', 'employee_id': 101, 'monthly_salary': 75000}
['FullTimeEmployee', 'Person', 'object']
Ananya (ID: 101)
```

Where Python looks for a method

`__mro__` (method resolution order) lists where Python searches, in order: the child class, then the parent, then `object` (the root of every class). `show_details` is not in `FullTimeEmployee`, so Python finds it in `Person`.

> Forget `super().__init__(...)` and the parent's attributes are never set:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Contractor(Person):
    def __init__(self, name, employee_id, hourly_rate):
        self.hourly_rate = hourly_rate          # forgot super().__init__(...)

contractor = Contractor("Rahul", 201, 1200)
print(contractor.__dict__)
contractor.show_details()
```

OUTPUT AND ERROR

```
{'hourly_rate': 1200}
AttributeError: 'Contractor' object has no attribute 'name'
```

21 Overriding and polymorphism

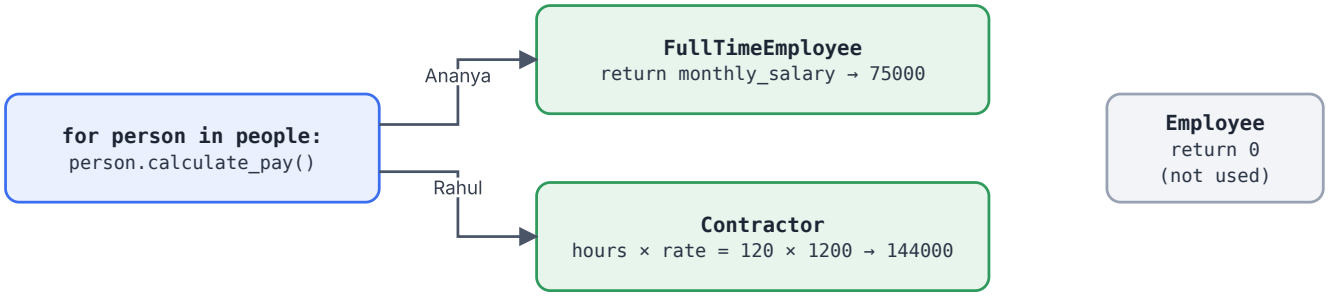


Figure 7.3 One call, person.calculate_pay(), runs a different method depending on the object's class.

A child can **override** a parent method: it defines a method with the same name, and its own version wins. Python checks the child class first (the `__mro__` order above).

NOTEBOOK CELL 5

```
class Employee:
    def __init__(self, name):
        self.name = name

    def calculate_pay(self):
        return 0  # Generic placeholder for this simple example

class FullTimeEmployee(Employee):
    def __init__(self, name, monthly_salary):
        super().__init__(name)
        self.monthly_salary = monthly_salary

    def calculate_pay(self):
        return self.monthly_salary

class Contractor(Employee):
    def __init__(self, name, hours_worked, hourly_rate):
        super().__init__(name)
        self.hours_worked = hours_worked
        self.hourly_rate = hourly_rate

    def calculate_pay(self):
        return self.hours_worked * self.hourly_rate

people = [
    FullTimeEmployee("Ananya", 75000),
    Contractor("Rahul", 120, 1200),
]

for person in people:
    print(f"{person.name}: ₹{person.calculate_pay():,}")
```

OUTPUT

```
Ananya: ₹75,000
Rahul: ₹144,000
```

The loop doesn't check "is this a contractor?". It just calls `calculate_pay()` and each object answers in its own way. That is **polymorphism** ("many forms"): one call, different behaviour.

22 ABC: a class that sets rules

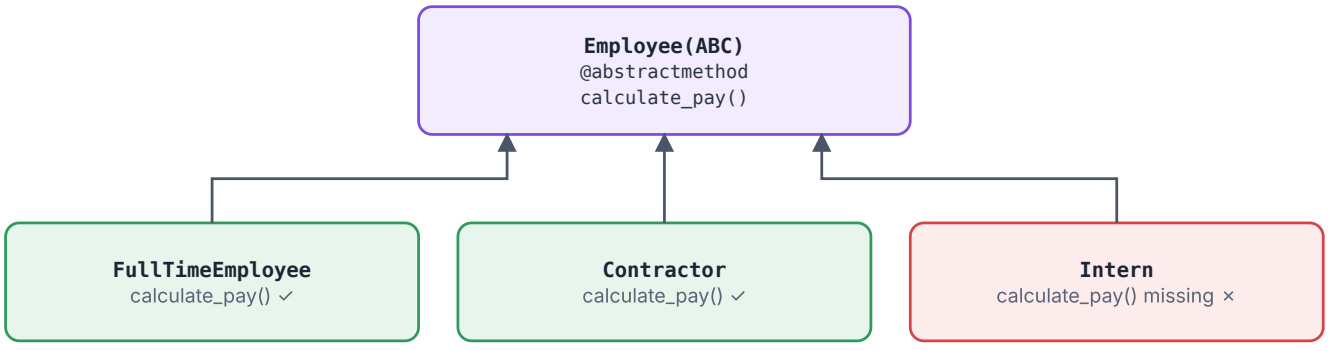


Figure 7.4 An ABC is a contract: every concrete child must implement calculate_pay(), or it cannot be created.

Sometimes every child **must** have a certain method — every employee type must say how it is paid. An **abstract base class** (ABC) states that rule:

- `class Employee(ABC):` marks it as abstract — a template, not something to create directly.
- `@abstractmethod` marks a method that every child must write.
- Python refuses to create an object of any class that still has an abstract method left unwritten.

NOTEBOOK CELL 8

```
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def calculate_pay(self):
        """Each employee type must define its own pay calculation."""
        pass

class FullTimeEmployee(Employee):
    def __init__(self, name, monthly_salary):
        super().__init__(name)
        self.monthly_salary = monthly_salary

    def calculate_pay(self):
        return self.monthly_salary

class Contractor(Employee):
    def __init__(self, name, hours_worked, hourly_rate):
        super().__init__(name)
        self.hours_worked = hours_worked
        self.hourly_rate = hourly_rate

    def calculate_pay(self):
        return self.hours_worked * self.hourly_rate

team = [
    FullTimeEmployee("Ananya", 75000),
    Contractor("Rahul", 120, 1200),
]

for person in team:
    print(person.name, "earns ₹", person.calculate_pay())
```

OUTPUT

```
Ananya earns ₹ 75000
Rahul earns ₹ 144000
```

What the ABC protects you from

The notebook's cell 10 has these lines commented out. Run, they give:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
employee = Employee("Meera")
```

ERROR

```
TypeError: Can't instantiate abstract class Employee without an implementation for abstract method
'calculate_pay'
```

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class Intern(Employee):
    pass

intern = Intern("Kiran")
```

ERROR

```
TypeError: Can't instantiate abstract class Intern without an implementation for abstract method
'calculate_pay'
```

Without the ABC, `Intern("Kiran")` would be created and the mistake would show up much later, when payroll calls `calculate_pay()`. With the ABC it fails at once, with a clear message.

23 A shared contract: customer notifications

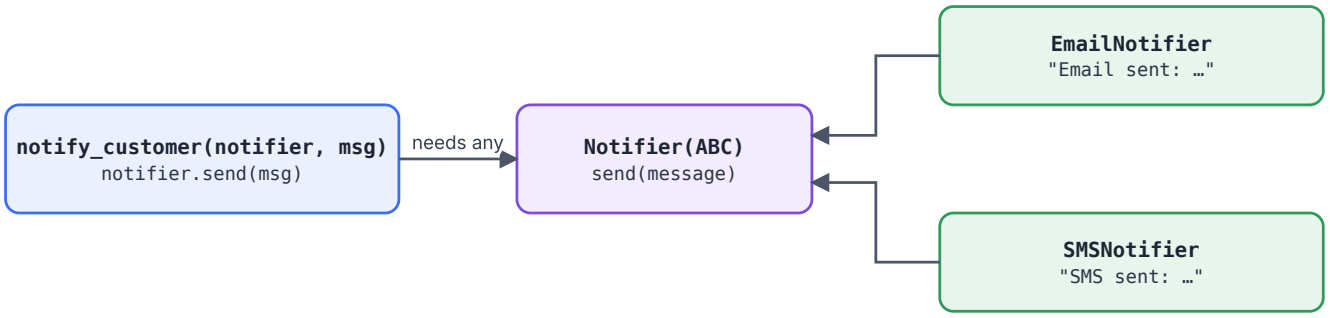


Figure 7.5 notify_customer depends only on the contract (send), not on which channel it gets.

ABCs are not only for employees. Every way of contacting a customer must support `send(message)`:

NOTEBOOK CELL 12

```
from abc import ABC, abstractmethod

class Notifier(ABC):
    @abstractmethod
    def send(self, message):
        pass

class EmailNotifier(Notifier):
    def send(self, message):
        print(f"Email sent: {message}")

class SMSNotifier(Notifier):
    def send(self, message):
        print(f"SMS sent: {message}")

def notify_customer(notifier, message):
    notifier.send(message)

notify_customer(EmailNotifier(), "Your order has shipped.")
notify_customer(SMSNotifier(), "Your order is ready for pickup.")
```

OUTPUT

```
Email sent: Your order has shipped.
SMS sent: Your order is ready for pickup.
```

Adding WhatsApp later means writing one new class, `WhatsAppNotifier(Notifier)`. `notify_customer` does not change. AI apps use the same idea to swap one LLM provider for another behind one interface.

24 Public, _internal and __mangled names

Python has no hard "private" like Java. It uses naming conventions:

Name	Means	Can outside code read it?
`name`	Public — normal use	Yes
`_points`	"Internal, please don't touch" — a convention	Yes, Python doesn't stop it
`__account_note`	Python renames it to `_CustomerAccount__account_note` (name mangling)	Not by its written name; yes by the new name

NOTEBOOK CELL 15

```
class CustomerAccount:
    def __init__(self, name, points):
        self.name = name
        self._points = points       # Convention: internal use
        self.__account_note = "VIP" # Name-mangled attribute

    def show_points(self):
        return self._points

customer = CustomerAccount("Meera", 120)
print(customer.name)
print(customer.show_points())
# customer.__account_note  # AttributeError: name is mangled
```

OUTPUT

```
Meera
120
```

What is really stored, and what happens when you try the commented line:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
class CustomerAccount:
    def __init__(self, name, points):
        self.name = name
        self._points = points
        self.__account_note = "VIP"

customer = CustomerAccount("Meera", 120)
print(customer.__dict__)                       # the real stored names
print(customer._points)                        # works — "_" is only a convention
print(customer._CustomerAccount__account_note) # works — mangling only renames
print(customer.__account_note)
```

OUTPUT AND ERROR

```
{'name': 'Meera', '_points': 120, '_CustomerAccount__account_note': 'VIP'}
120
VIP
AttributeError: 'CustomerAccount' object has no attribute '__account_note'
```

Name mangling only avoids accidental name clashes in child classes. It is **not** security. Never keep API keys or passwords in objects that get printed or logged.

25 @classmethod and @staticmethod

Kind	First parameter	Called as	Typical use
Instance method	`self` — one object	`customer.greet()`	Work with one object's data
`@classmethod`	`cls` — the class itself	`Customer.from_record(record)`	Another way to create objects (from a dict, a DB row, JSON)
`@staticmethod`	nothing	`Customer.is_valid_email(email)`	A helper that belongs with the class but needs no object

NOTEBOOK CELL 17

```
class Customer:
    def __init__(self, name, email):
        self.name = name
        self.email = email

    @classmethod
    def from_record(cls, record):
        # Build a Customer from a dictionary
        return cls(record["name"], record["email"])

    @staticmethod
    def is_valid_email(email):
        # Simple teaching example, not full email validation
        return "@" in email and "." in email.split("@")[-1]

record = {"name": "Meera", "email": "meera@example.com"}
customer = Customer.from_record(record)
print(customer.name)
print(Customer.is_valid_email(customer.email))
```

OUTPUT

```
Meera
True
```

`cls(record["name"], record["email"])` is the same as `Customer(…)`, but written with `cls` it also works for child classes. `from_record` is useful when data arrives as dicts — from JSON (Day 04) or a database (Day 08).

26 Composition: HAS-A instead of IS-A

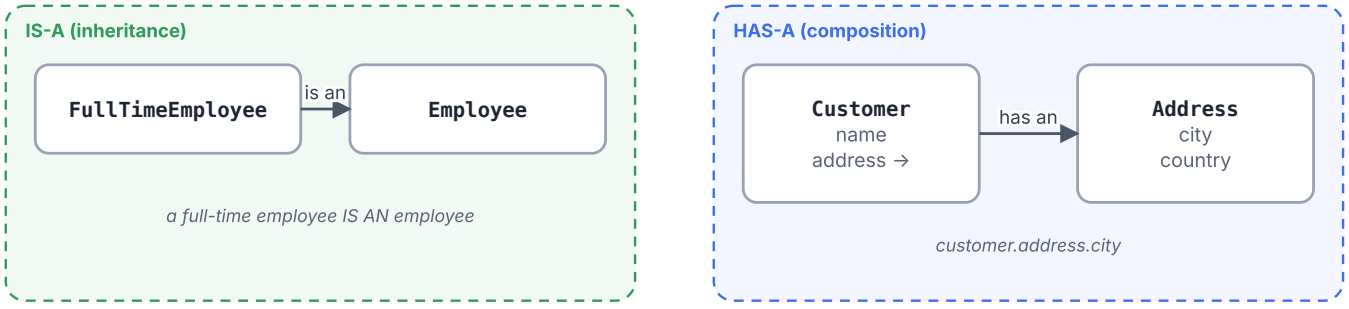


Figure 7.6 IS-A → inheritance. HAS-A → composition: one object stores another as an attribute.

Not every relationship is inheritance. An address is not a kind of customer — a customer **has** an address. Store the `Address` object as an attribute of the `Customer`:

NOTEBOOK CELL 19

```
class Address:
    def __init__(self, city, country):
        self.city = city
        self.country = country

class Customer:
    def __init__(self, name, address):
        self.name = name
        self.address = address  # Customer HAS-A Address

address = Address("Hyderabad", "India")
customer = Customer("Meera", address)

print(customer.name)
print(customer.address.city)
print(customer.address.country)
```

OUTPUT

```
Meera
Hyderabad
India
```

Ask	If yes, use	Example
Is B a kind of A?	Inheritance: `class B(A)`	Contractor is an Employee
Does A have / use a B?	Composition: `self.b = b`	Customer has an Address; a service has a Notifier

When in doubt, choose composition. Deep inheritance chains are hard to follow; small objects that hold other objects are easier to change.

27 isinstance(): check an object's type

NOTEBOOK CELL 21

```
class Employee:
    pass

class FullTimeEmployee(Employee):
    pass

employee = FullTimeEmployee()

print(isinstance(employee, FullTimeEmployee))  # True
print(isinstance(employee, Employee))          # True: parent class too
print(isinstance(employee, str))               # False
```

OUTPUT

```
True
True
False
```

`isinstance` follows the IS-A chain, so a `FullTimeEmployee` is also an `Employee`. Prefer it over `type(x) == Employee`, which would say False for the child.

28 Day 07 recap and practice

Concept	Remember
Inheritance	`class Child(Parent)` — reuse shared attributes and methods (IS-A)
`super().__init__()`	Lets the parent fill in its attributes; still one object
Method lookup	Child first, then parent, then `object` (`__mro__`)
Overriding	Child defines a method with the same name; its version wins
Polymorphism	One call, `calculate_pay()`, different behaviour per class
ABC / `@abstractmethod`	A contract: children must implement the method or can't be created
`_x` / `__x`	Convention / renamed — neither is security
`@classmethod` / `@staticmethod`	Gets `cls` (alternative constructors) / gets nothing (helpers)
Composition	HAS-A: store another object as an attribute
`isinstance`	True for the class and its parents

Practice (from the notebook)

1. Create a `Customer` base class with `name` and `email`; create `PremiumCustomer` that adds a discount percentage.
2. Override a method called `get_discount()` in `PremiumCustomer`.
3. Create an ABC called `PaymentMethod` with abstract method `pay(amount)`. Implement `CardPayment` and `UPIPayment`.
4. Create a function that accepts any `PaymentMethod` and calls `pay(amount)` without checking which subclass it received.
5. Add a `@classmethod` alternative constructor that creates a customer from a dictionary.
6. Model a `Customer` that HAS-A `Address` using composition.

DAY 08

Pydantic and Databases

Data from APIs, LLMs and databases can't be trusted blindly. Pydantic checks it against a class you write. Then: how Python reads and writes PostgreSQL rows safely, and validates them on the way in.

Notebook: **08_Pydantic_Database.ipynb**

Why validate | BaseModel | ValidationError | Field rules | Optional & Literal | dict / JSON in & out | Validate LLM output | Nested models | Database basics | Parameters vs SQL injection | Row → model

29 Why validate data?



Figure 8.1 A Pydantic model is a gate between untrusted data and your application.

A customer record arrives from an API or an LLM. The ID should be a number, but it came as text, and nothing checks that the required fields are there:

NOTEBOOK CELL 2

```
customer_data = {
    "customer_id": "1001",
    "name": "Ravi Kumar",
    "email": "ravi@example.com"
}

print(customer_data["name"])
print("ID type:", type(customer_data["customer_id"]).__name__)
```

OUTPUT

```
Ravi Kumar
ID type: str
```

A normal Day 06 class doesn't check anything either. **Pydantic** is a library (`pip install pydantic`) whose models check every value when the object is created:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
from pydantic import BaseModel, ValidationError

class PlainCustomer:                        # a normal Day 06 class
    def __init__(self, customer_id, name):
        self.customer_id = customer_id
        self.name = name

class Customer(BaseModel):                  # same idea, but inherits from BaseModel
    customer_id: int
    name: str

plain = PlainCustomer("not-a-number", "Ravi Kumar")
print("Plain class accepted:", repr(plain.customer_id))   # wrong data slips in

try:
    Customer(customer_id="not-a-number", name="Ravi Kumar")
except ValidationError as error:
    print("Pydantic refused:", error.errors()[0]["msg"])

print(issubclass(Customer, BaseModel))     # Day 07: Customer IS-A BaseModel
```

OUTPUT

```
Plain class accepted: 'not-a-number'
Pydantic refused: Input should be a valid integer, unable to parse string as an integer
True
```

Everything from Days 06 and 07 applies: a Pydantic model is a **class** that **inherits** from `BaseModel`. You get objects, attributes and the dot — plus automatic checking, with no `__init__` to write.

30 Your first model

List the fields with their types (the type hints from Day 01). Pydantic writes the `__init__` for you, using these hints as rules:

NOTEBOOK CELL 4

```
from pydantic import BaseModel, Field, ValidationError
from typing import Literal
```

NOTEBOOK CELL 6

```
class Customer(BaseModel):
    customer_id: int
    name: str
    email: str

customer = Customer(
    customer_id="1001",
    name="Ravi Kumar",
    email="ravi@example.com"
)

print(customer)
print("ID:", customer.customer_id)
print("ID type:", type(customer.customer_id).__name__)
```

OUTPUT

```
customer_id=1001 name='Ravi Kumar' email='ravi@example.com'
ID: 1001
ID type: int
```

`"1001"` is text, but it clearly means the number 1001, so Pydantic **converts** it. Text that can't be a number is refused with a `ValidationError`:

NOTEBOOK CELL 8

```
try:
    invalid_customer = Customer(
        customer_id="not-a-number",
        name="Ravi Kumar",
        email="ravi@example.com"
    )
except ValidationError as error:
    print(error)
```

OUTPUT

```
1 validation error for Customer
customer_id
  Input should be a valid integer, unable to parse string as an integer [type=int_parsing,
input_value='not-a-number', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/int_parsing
```

How to read a ValidationError

Line	Tells you
`1 validation error for Customer`	How many fields failed, and in which model
`customer_id`	Which field
`Input should be a valid integer…`	What was wrong, in plain words
`[type=int_parsing, input_value='not-a-number', …]`	The error code and the exact value you sent

A missing field is caught too — every field without a default is required:

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
try:
    Customer(name="Ravi Kumar")            # customer_id left out
except ValidationError as error:
    print(error)
```

OUTPUT

```
1 validation error for Customer
customer_id
  Field required [type=missing, input_value={'name': 'Ravi Kumar'}, input_type=dict]
    For further information visit https://errors.pydantic.dev/2.13/v/missing
```

31 Add rules with Field

A type alone allows `-1` as an ID or `""` as a name. `Field(...)` adds limits. A plain value after `=` is a default:

NOTEBOOK CELL 10

```
class CustomerRecord(BaseModel):
    customer_id: int = Field(gt=0)
    name: str = Field(min_length=1, max_length=60)
    email: str = Field(min_length=5)
    city: str = "Hyderabad"

good_customer = CustomerRecord(
    customer_id=1001,
    name="Ravi Kumar",
    email="ravi@example.com"
)

print(good_customer)
print("City default:", good_customer.city)
```

OUTPUT

```
customer_id=1001 name='Ravi Kumar' email='ravi@example.com' city='Hyderabad'
City default: Hyderabad
```

Break all three rules at once and Pydantic reports **every** problem, not just the first. `error.errors()` gives them as a list of dicts you can loop over:

NOTEBOOK CELL 11

```
try:
    CustomerRecord(
        customer_id=-1,
        name="",
        email="bad"
    )
except ValidationError as error:
    for item in error.errors():
        print("Field:", item["loc"], "| Problem:", item["msg"])
```

OUTPUT

```
Field: ('customer_id',) | Problem: Input should be greater than 0
Field: ('name',) | Problem: String should have at least 1 character
Field: ('email',) | Problem: String should have at least 5 characters
```

Rule	Meaning
`gt=0` / `ge=0`	Greater than 0 / 0 or greater
`lt=` / `le=`	Less than / less than or equal
`min_length=1` / `max_length=60`	Text length limits
`city: str = "Hyderabad"`	Default used when the field is left out

The notebook stores email as plain `str` to keep things simple. For real email checking, Pydantic has `EmailStr` (install with `pip install "pydantic[email]"`).

32 Optional fields and allowed values

NOTEBOOK CELL 14

```
class Order(BaseModel):
    order_id: int = Field(gt=0)
    customer_id: int = Field(gt=0)
    total_amount: float = Field(ge=0)
    status: Literal["Pending", "Shipped", "Delivered", "Cancelled"]
    notes: str | None = None

order = Order(
    order_id=5001,
    customer_id=1001,
    total_amount=2499.50,
    status="Shipped"
)

print(order)
print("Notes:", order.notes)
```

OUTPUT

```
order_id=5001 customer_id=1001 total_amount=2499.5 status='Shipped' notes=None
Notes: None
```

Write	Means
`notes: str | None = None`	May be text or `None`; optional because it has a default
`status: Literal["Pending", "Shipped", …]`	Only these exact values are allowed
`total_amount: float = Field(ge=0)`	A number, 0 or more

NOTEBOOK CELL 15

```
try:
    Order(
        order_id=5002,
        customer_id=1001,
        total_amount=2499.50,
        status="On the way"  # not one of the allowed values
    )
except ValidationError as error:
    print(error)
```

OUTPUT

```
1 validation error for Order
status
  Input should be 'Pending', 'Shipped', 'Delivered' or 'Cancelled' [type=literal_error,
input_value='On the way', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/literal_error
```

`Literal` is ideal for LLM output: a category, a status, a confidence level. The model must answer with one of your words — `"On the way"` is refused.

33 Validate dicts and JSON, and convert back

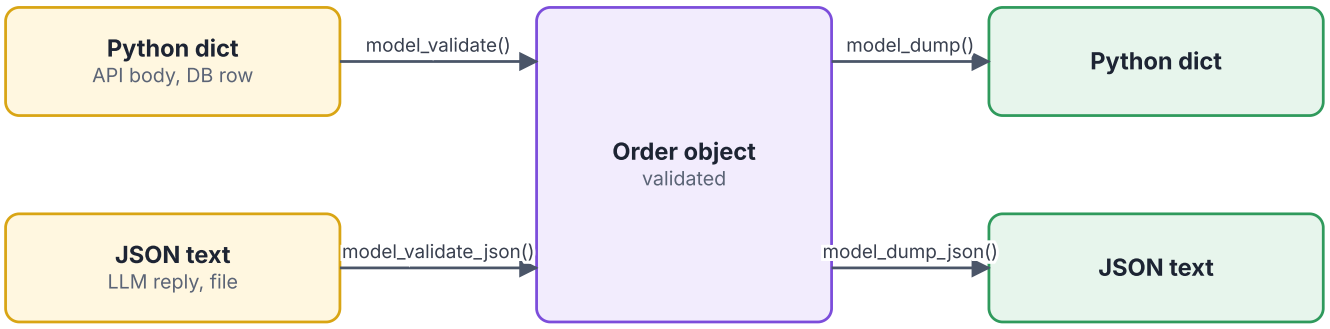


Figure 8.2 Four methods move data in and out of a model — Day 04's loads / dumps, with checking built in.

Data rarely arrives as keyword arguments. It arrives as a dict or as JSON text. Note how the text values `"5003"` and `"799.99"` become an `int` and a `float`:

NOTEBOOK CELL 18

```
# Validate a Python dictionary, such as an API or database record.
raw_order = {
    "order_id": "5003",
    "customer_id": "1001",
    "total_amount": "799.99",
    "status": "Pending"
}

order_from_dict = Order.model_validate(raw_order)
print(order_from_dict)
print(type(order_from_dict.total_amount).__name__)
```

OUTPUT

```
order_id=5003 customer_id=1001 total_amount=799.99 status='Pending' notes=None
float
```

NOTEBOOK CELL 19

```
# Validate JSON text in one step.
order_json = '''
{
  "order_id": 5004,
  "customer_id": 1002,
  "total_amount": 1299.0,
  "status": "Delivered",
  "notes": "Leave at reception"
}
'''

order_from_json = Order.model_validate_json(order_json)
print(order_from_json)
```

OUTPUT

```
order_id=5004 customer_id=1002 total_amount=1299.0 status='Delivered' notes='Leave at reception'
```

Going the other way — to save, log or send the data:

NOTEBOOK CELL 21

```
order_dict = order_from_json.model_dump()
order_json_output = order_from_json.model_dump_json()

print(type(order_dict).__name__, order_dict)
print(type(order_json_output).__name__, order_json_output)
```

OUTPUT

```
dict {'order_id': 5004, 'customer_id': 1002, 'total_amount': 1299.0, 'status': 'Delivered', 'notes':
'Leave at reception'}
str {"order_id":5004,"customer_id":1002,"total_amount":1299.0,"status":"Delivered","notes":"Leave at
reception"}
```

Day 04 (no checks)	Day 08 (with checks)
`json.loads(text)` → dict	`Order.model_validate_json(text)` → checked object
use the dict as it is	`Order.model_validate(dict)` → checked object

Day 04 (no checks)	Day 08 (with checks)
`json.dumps(dict)` → text	`order.model_dump_json()` → text

34 AI use case: validate an LLM's classification

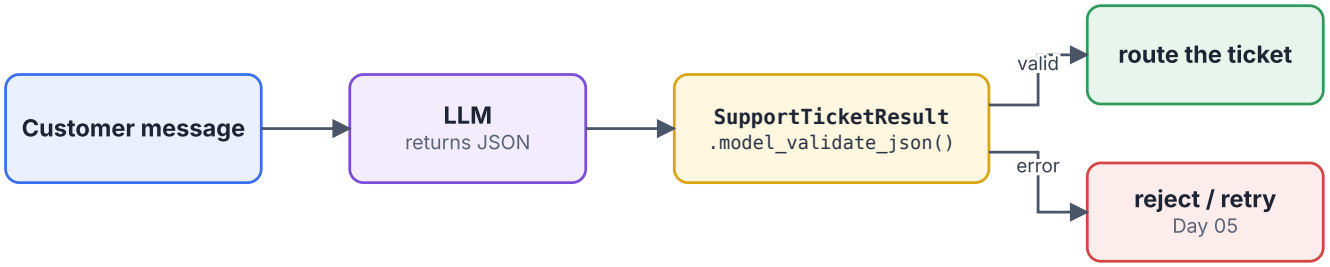


Figure 8.3 Never use LLM output directly. Validate it first; reject or retry when it doesn't fit.

You ask an LLM to classify a support message as JSON. It usually behaves, but sometimes invents a category or skips a field. Describe the answer you expect as a model and validate it:

NOTEBOOK CELL 23

```
class SupportTicketResult(BaseModel):
    category: Literal["TRACK_ORDER", "CANCEL_ORDER", "REFUND", "GENERAL"]
    confidence: Literal["low", "medium", "high"]
    reason: str = Field(min_length=5)

ai_json = '''
{
  "category": "TRACK_ORDER",
  "confidence": "high",
  "reason": "The customer asks where their order is."
}
'''

ticket_result = SupportTicketResult.model_validate_json(ai_json)
print("Category:", ticket_result.category)
print("Confidence:", ticket_result.confidence)
print("Reason:", ticket_result.reason)
```

OUTPUT

```
Category: TRACK_ORDER
Confidence: high
Reason: The customer asks where their order is.
```

Validation checks the **shape and the rules**, not the truth. A valid `TRACK_ORDER` can still be the wrong category for that message.

35 Nested models

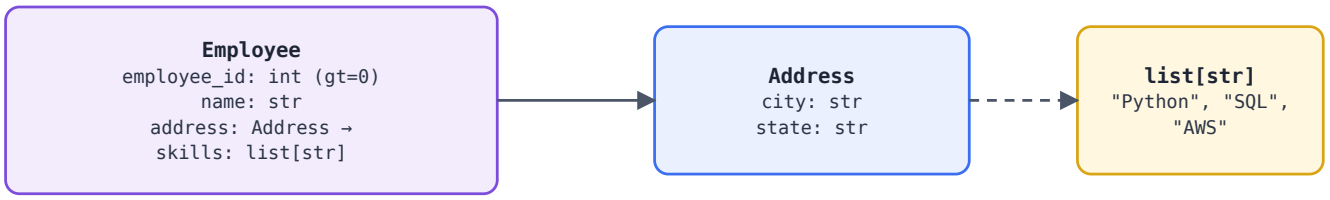


Figure 8.4 Models inside models. Pydantic checks the inner Address and every item of the list too.

A field's type can be another model — composition (Day 07) with validation. Nested dicts become nested objects:

NOTEBOOK CELL 26

```
class Address(BaseModel):
    city: str
    state: str

class Employee(BaseModel):
    employee_id: int = Field(gt=0)
    name: str
    address: Address
    skills: list[str]

employee_data = {
    "employee_id": 101,
    "name": "Ananya",
    "address": {"city": "Hyderabad", "state": "TS"},
    "skills": ["Python", "SQL", "AWS"]
}

employee = Employee.model_validate(employee_data)
print(employee.name)
print(employee.address.city)
print(employee.skills)
```

OUTPUT

```
Ananya
Hyderabad
['Python', 'SQL', 'AWS']
```

`employee.address.city` — the dot works at every level, just like `customer.address.city` on Day 07.

36 Database basics: where Pydantic fits

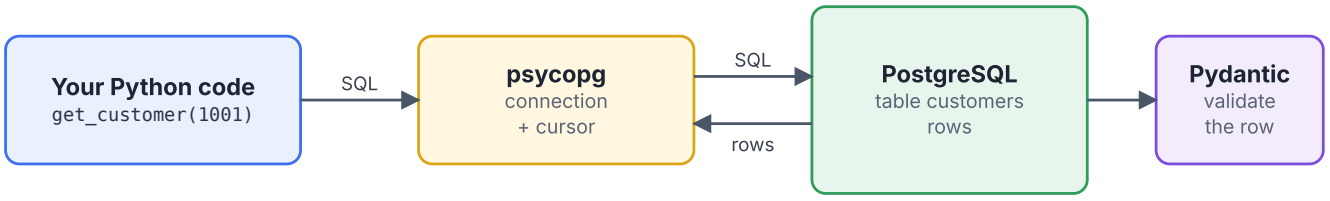


Figure 8.5 Python talks to PostgreSQL through a driver (psycopg). SQL goes in, rows come back.

Word	Meaning
Connection	An open session with the database server
Cursor	The object that runs SQL and reads results
`INSERT` / `SELECT`	Add a row / read rows
Commit	Save the changes made in a transaction (psycopg commits when the `with` block ends without error)
Parameterised query	Values sent separately from the SQL text — prevents SQL injection

The flow for reading: **database row → Python dict → Pydantic validation → your application logic**.

37 Setting up PostgreSQL (optional)

The rest of Day 08 needs a running PostgreSQL server. Install it from postgresql.org/download (Windows and Mac installers), then create a database called `shopsmart` — in pgAdmin, or with `psql -U postgres -c "CREATE DATABASE shopsmart;"`. With your `venv` active (Day 01), install the packages:

Step	Mac	Windows (PowerShell)
Activate the venv	`source venv/bin/activate`	`venv\Scripts\activate`
Install packages	`pip install pydantic "psycopg[binary]" python-dotenv`	`pip install pydantic "psycopg[binary]" python-dotenv`
Create `.env` next to the notebook	`touch .env`, then edit it	`New-Item .env`, then edit it

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=shopsmart
DB_USER=postgres
DB_PASSWORD=your_password
```

`.env` holds your password. Add it to `.gitignore` and never commit it to Git. `load_dotenv()` reads it into environment variables so the code never contains the password.

38 Connect, create a table, insert a row

A helper that refuses to start without settings, then opens a connection:

NOTEBOOK CELL 29

```
# Optional: PostgreSQL connection helper.
import os
from dotenv import load_dotenv
import psycopg

load_dotenv()

def get_connection():
    required = ["DB_NAME", "DB_USER", "DB_PASSWORD"]
    missing = [key for key in required if not os.getenv(key)]
    if missing:
        raise ValueError(f"Missing database settings: {', '.join(missing)}")

    return psycopg.connect(
        host=os.getenv("DB_HOST", "localhost"),
        port=int(os.getenv("DB_PORT", "5432")),
        dbname=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
    )

print("Connection helper defined. Connect only when PostgreSQL is configured.")
```

OUTPUT

```
Connection helper defined. Connect only when PostgreSQL is configured.
```

Each function opens a connection in a `with` block (Day 04): changes are committed and the connection is closed when the block ends. In the notebook the calls at the bottom are commented out; the outputs here come from running them against a local PostgreSQL.

NOTEBOOK CELL 30

```
# Optional: create a small customers table.
# Run this cell only when your PostgreSQL database is ready.
def create_customers_table():
    with get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute("""
                CREATE TABLE IF NOT EXISTS customers (
                    customer_id INTEGER PRIMARY KEY,
                    name VARCHAR(60) NOT NULL,
                    email VARCHAR(120) NOT NULL,
                    city VARCHAR(60) NOT NULL
                )
            """)
    print("Customers table is ready.")

# Uncomment to run:
create_customers_table()
```

OUTPUT

```
Customers table is ready.
```

NOTEBOOK CELL 31

```
# Optional: insert a customer using parameters, not an f-string.
def insert_customer(customer_id, name, email, city):
    with get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO customers (customer_id, name, email, city)
                VALUES (%s, %s, %s, %s)
                ON CONFLICT (customer_id) DO NOTHING
                """,
                (customer_id, name, email, city)
            )
    print("Insert checked for customer:", customer_id)

# Uncomment to run after creating the table:
insert_customer(1001, "Ravi Kumar", "ravi@example.com", "Hyderabad")
```

OUTPUT

```
Insert checked for customer: 1001
```

`ON CONFLICT (customer_id) DO NOTHING` makes the insert safe to run twice: the second time the row already exists and nothing happens.

39 Parameters, not f-strings: SQL injection

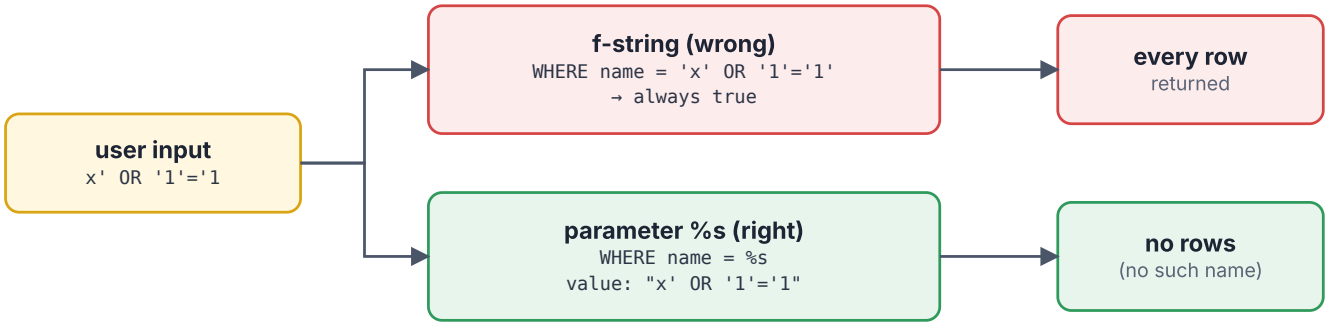


Figure 8.6 Glued into the SQL, the input changes the query. Sent as a parameter, it stays just a value.

The notebook passes values with `%s` placeholders and a tuple, never with an f-string. Here is why — run against the same database (after adding a second customer, 1002 Ananya Sharma):

EXTRA EXAMPLE (NOT IN THE NOTEBOOK)

```
name_input = "x' OR '1'='1"      # what a user (or an LLM) might type

with get_connection() as conn, conn.cursor() as cur:
    # WRONG: the value is glued into the SQL text
    sql = f"SELECT customer_id, name FROM customers WHERE name = '{name_input}'"
    print("SQL sent :", sql)
    cur.execute(sql)
    print("f-string :", cur.fetchall())

    # RIGHT: the value travels separately from the SQL
    cur.execute("SELECT customer_id, name FROM customers WHERE name = %s", (name_input,))
    print("parameter:", cur.fetchall())
```

OUTPUT

```
SQL sent : SELECT customer_id, name FROM customers WHERE name = 'x' OR '1'='1'
f-string : [(1001, 'Ravi Kumar'), (1002, 'Ananya Sharma')]
parameter: []
```

With the f-string, the quote inside the input closed the SQL string early and `OR '1'='1'` became part of the query, so it matched every customer. As a parameter, the whole input is compared as one name — no match.

This matters even more in AI apps: if an LLM builds part of a query from user text, a user can steer it. Always pass values as parameters: `cur.execute("… WHERE id = %s", (customer_id,))`. Note the comma — `(customer_id,)` is a one-item tuple (Day 01).

40 Read a row and validate it

`fetchone()` returns one row as a tuple, or `None` if nothing matched. The function turns it into a dict, and `CustomerRecord.model_validate` checks it before the app uses it:

NOTEBOOK CELL 32

```
# Optional: fetch a row as a dictionary, then validate it.
def get_customer(customer_id):
    with get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT customer_id, name, email, city FROM customers WHERE customer_id = %s",
                (customer_id,)
            )
            row = cur.fetchone()

    if row is None:
        return None

    return {
        "customer_id": row[0],
        "name": row[1],
        "email": row[2],
        "city": row[3]
    }

# Uncomment to query your database:
row = get_customer(1001)
if row is not None:
    validated_customer = CustomerRecord.model_validate(row)
    print(validated_customer)
```

OUTPUT

```
customer_id=1001 name='Ravi Kumar' email='ravi@example.com' city='Hyderabad'
```

Step	Value
`row`	`(1001, 'Ravi Kumar', 'ravi@example.com', 'Hyderabad')` — a tuple
dict	`{'customer_id': 1001, 'name': 'Ravi Kumar', …}`
model	`CustomerRecord` object — fields checked: id > 0, name 1–60 characters, ...

41 Day 08 recap and practice

Concept	Remember
`BaseModel`	A class whose fields are checked when an object is created
Type hints	Describe each field; compatible values are converted (`"1001"` → 1001)
`Field(...)`	Limits like `gt=0` , `min_length=1` ; plain `= value` is a default
`str | None = None`	Optional field
`Literal[...]`	Only the listed values are allowed — great for LLM categories
`ValidationError`	Lists every failed field, the reason and the input
`model_validate` / `_json`	dict / JSON text → checked object
`model_dump` / `_json`	object → dict / JSON text

Concept	Remember
Database	connection → cursor → `execute` → `fetchone` / commit
SQL parameters	`%s` + tuple; never build SQL with f-strings

Practice (from the notebook)

1. Create a `Product` model with `product_id`, `name`, `price`, and `stock`.
2. Require `price` to be greater than zero and `stock` to be zero or greater.
3. Add a `status` field to an `Order` that allows only `Pending`, `Shipped`, or `Delivered`.
4. Validate an order from a dictionary where numeric values arrive as strings.
5. Create a `SupportTicketResult` with an invalid category and inspect the error.
6. Create an `Employee` model with a nested `Address` and a list of skills.
7. Explain why SQL values should be passed as parameters rather than inserted into a query using an f-string.

WRAP-UP

Days 06–08 in one script

A small support-ticket service: a class that HAS-A notifier, loads a customer from PostgreSQL, validates the row and the LLM's JSON with Pydantic, and only then notifies the customer.

Notebook: **support_service.py (new — not in the notebooks)**

Day 06 class + state | Day 07 ABC + composition | Day 08 Pydantic + SQL parameters

42 The script

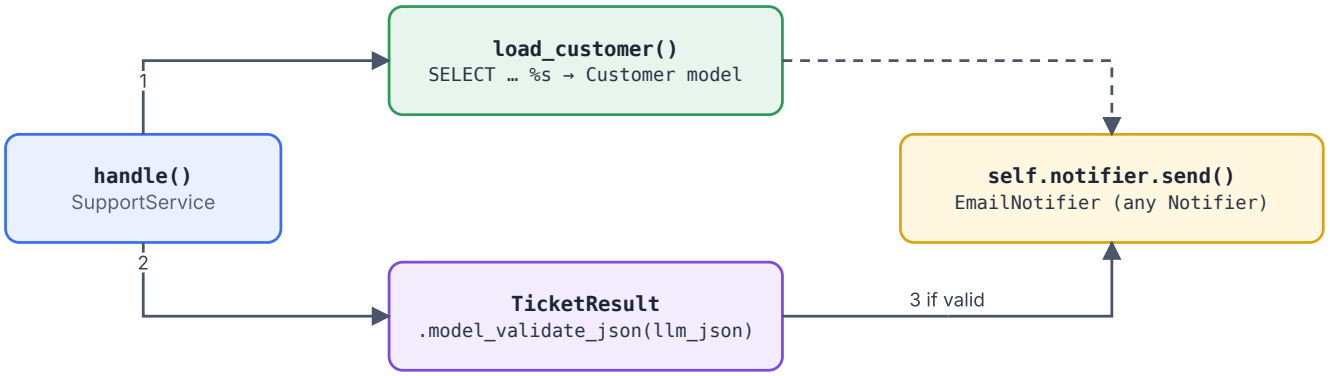


Figure 9.1 What service.handle(customer_id, llm_json) does.

Save it as `support_service.py` in the folder that holds your `.env`, then run `python support_service.py` (Mac: `python3`). It uses the `customers` table created on Day 08 (customers 1001 and 1002).

```
import os
from abc import ABC, abstractmethod
from typing import Literal

import psycopg
from dotenv import load_dotenv
from pydantic import BaseModel, Field, ValidationError

load_dotenv()


class Customer(BaseModel):                          # Day 08: validated record
    customer_id: int = Field(gt=0)
    name: str = Field(min_length=1)
    email: str
    city: str


class TicketResult(BaseModel):                      # Day 08: validates the LLM's JSON
    category: Literal["TRACK_ORDER", "CANCEL_ORDER", "REFUND", "GENERAL"]
    reply: str = Field(min_length=5)


class Notifier(ABC):                                # Day 07: the contract
    @abstractmethod
    def send(self, customer, message):
```

```
        pass


class EmailNotifier(Notifier):                    # Day 07: one implementation
    def send(self, customer, message):
        print(f"Email to {customer.email}: {message}")


class SupportService:                             # Day 06: data + behaviour together
    def __init__(self, notifier):
        self.notifier = notifier                  # Day 07: HAS-A notifier
        self.handled = 0

    def load_customer(self, customer_id):
        with psycopg.connect(host=os.getenv("DB_HOST"), port=int(os.getenv("DB_PORT")),
                             dbname=os.getenv("DB_NAME"), user=os.getenv("DB_USER"),
                             password=os.getenv("DB_PASSWORD")) as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT customer_id, name, email, city FROM customers "
                            "WHERE customer_id = %s", (customer_id,))
                row = cur.fetchone()
        if row is None:
            return None
        return Customer.model_validate(
            {"customer_id": row[0], "name": row[1], "email": row[2], "city": row[3]})

    def handle(self, customer_id, llm_json):
        customer = self.load_customer(customer_id)
        if customer is None:
            print(f"No customer with id {customer_id}")
            return
        try:
            result = TicketResult.model_validate_json(llm_json)
        except ValidationError as error:
            print(f"Rejected LLM output for {customer.name}: {error.errors()[0]['msg']}")
            return
        self.notifier.send(customer, result.reply)
        self.handled += 1


service = SupportService(EmailNotifier())
service.handle(1001, '{"category": "TRACK_ORDER", "reply": "Your order ORD-3042 is in transit."}')
service.handle(1002, '{"category": "OTHER", "reply": "Not sure."}')
service.handle(9999, '{"category": "GENERAL", "reply": "Hello there."}')
print("Tickets handled:", service.handled)
```

OUTPUT

```
Email to ravi@example.com: Your order ORD-3042 is in transit.
Rejected LLM output for Ananya Sharma: Input should be 'TRACK_ORDER', 'CANCEL_ORDER', 'REFUND' or
'GENERAL'
No customer with id 9999
Tickets handled: 1
```

43 Read it like this

Code	Idea	Day
`class SupportService` with `self.handled`	An object keeps state between calls	06

Code	Idea	Day
`self.notifier = notifier`	Composition — the service HAS-A notifier	07
`class Notifier(ABC)` + `EmailNotifier`	Contract + one implementation; swap in SMS without touching the service	07
`WHERE customer_id = %s` , `(customer_id,)`	Parameterised query	08
`Customer.model_validate({...})`	Database row → dict → checked object	08
`TicketResult.model_validate_json(llm_json)`	LLM output checked before use; `OTHER` is rejected	08
`try / except ValidationError`	Bad LLM output is handled, not a crash	05

Try: add an `SMSNotifier` and pass it to `SupportService` ; add a `"REFUND"` ticket for 1002; remove `reply` from the JSON and read the error.